

MOBILE APPLICATION DEVELOPMENT

Course Notes

Week 10 & Week 11

App Testing, Debugging & Deployment

Lecture 19 — App Testing and Debugging: Unit & Widget Testing in Flutter

Lecture 20 — Testing in React Native: Jest & React Native Testing Library

Lecture 21 — App Deployment: Building APKs and IPAs

Lecture 22 — Publishing Apps on Google Play and Apple App Store

Table of Contents

Table of Contents	2
Week 10	4
Lecture 19: App Testing and Debugging	4
1. Introduction to App Testing	4
1.1 The Three Levels of Testing in Flutter	4
2. Unit Testing in Flutter	4
2.1 Why Unit Testing Matters	5
2.2 Structure of a Unit Test	5
2.3 Code Example: Unit Test	5
3. Widget Testing in Flutter	6
3.1 Key Concepts in Widget Testing	6
3.2 Code Example: Widget Test	7
3.3 Unit Testing vs Widget Testing	8
4. Debugging in Flutter	8
Lecture 20: Testing in React Native	10
1. Introduction to Testing in React Native	10
2. Jest: The JavaScript Testing Framework	10
2.1 Core Features of Jest	10
2.2 Code Example: Jest Unit Test	10
3. React Native Testing Library (RNTL)	11
3.1 Guiding Philosophy	11
3.2 Code Example: Component Test with RNTL	12
3.3 Comparing Flutter Testing and React Native Testing	13
Week 11	14
Lecture 21: App Deployment	14
1. What Is App Deployment?	14
2. Preparing an App for Deployment	14
3. Building an APK for Android	14
3.1 App Bundles (AAB) vs APK	15
3.2 Code Signing on Android	15
3.3 Building the APK	15
4. Building an IPA for iOS	15
4.1 The Apple Developer Program and Certificates	16

4.2 Building the IPA.....	16
4.3 APK vs IPA: Side-by-Side Comparison	16
Lecture 22: Publishing Apps on Google Play and Apple App Store	18
1. Introduction to App Store Publishing	18
2. Publishing on Google Play (Google Play Console).....	18
2.1 Steps to Publish on Google Play.....	18
2.2 Release Tracks.....	18
3. Publishing on the Apple App Store (App Store Connect).....	19
3.1 Steps to Publish on the Apple App Store	19
3.2 Common Reasons for App Store Rejection.....	19
3.3 Google Play vs Apple App Store: Comparison.....	20
4. Post-Publishing Responsibilities.....	20
Assignment No. 3	21
Instructions	21
Section A: Short Conceptual Questions (5 marks each — 20 marks)	21
Section B: Analytical Questions (10 marks each — 20 marks).....	21
Section C: Case Study Question (10 marks)	21
Marking Rubric.....	22

Week 10

Lecture 19: App Testing and Debugging

Topic: Unit Testing and Widget Testing in Flutter

1. Introduction to App Testing

Before any mobile application is released to real users, it must be verified to behave correctly under a wide range of conditions. App testing is the systematic process of evaluating a mobile application to confirm that it functions as intended, is free of critical defects, and delivers a stable experience across devices. Testing and debugging are not optional extras in professional app development — they are core engineering practices that protect both the user experience and the reputation of the development team.

In Flutter development, testing is treated as a first-class citizen of the framework. Flutter ships with a complete testing toolkit out of the box, which allows developers to write and run tests without installing extra libraries. This toolkit is organised around three main categories of tests.

1.1 The Three Levels of Testing in Flutter

Test Type	What It Checks	Speed	Example
Unit Test	A single function, method, or class in isolation	Very fast (milliseconds)	Testing a function that calculates tax on a price
Widget Test	A single widget's UI and behaviour in a simulated environment	Fast (no real device needed)	Testing that a Login button is disabled when fields are empty
Integration Test	The complete app, or a large part of it, running on a real device or emulator	Slower (full app)	Testing the entire checkout flow from cart to payment confirmation

2. Unit Testing in Flutter

DEFINITION: Unit Test

A unit test verifies the correctness of a single unit of code — typically a function, method, or class — completely isolated from the rest of the application, including the UI, databases, and network calls. Its purpose is to confirm that a specific piece of business logic produces the expected output for a given input.

Unit tests in Flutter are written using the built-in `flutter_test` package, which is itself built on top of Dart's test package. Because unit tests do not touch the UI, they are extremely fast to run and are

usually executed first whenever code changes are made — a practice often described as the foundation of the “testing pyramid”, where unit tests form the largest and fastest layer of a test suite.

2.1 Why Unit Testing Matters

- Catches logic errors early, before they reach the user interface or production.
- Acts as living documentation — a new developer can read the tests to understand what a function is supposed to do.
- Makes refactoring safe; if a test suite still passes after changing the internal code, the developer can be confident nothing was broken.
- Reduces the cost of fixing bugs — a defect caught during development costs far less to fix than one discovered after release.

2.2 Structure of a Unit Test

A typical Flutter unit test follows the Arrange–Act–Assert (AAA) pattern:

1. Arrange — set up the objects and data needed for the test.
2. Act — call the function or method being tested.
3. Assert — check that the result matches what was expected, using the `expect()` function.

2.3 Code Example: Unit Test

Suppose we have a simple calculator class in our application:

```
// lib/calculator.dart
class Calculator {
  int add(int a, int b) {
    return a + b;
  }

  double calculateDiscount(double price, double percentage) {
    return price - (price * percentage / 100);
  }
}
```

The corresponding unit test file looks like this:

```
// test/calculator_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/calculator.dart';

void main() {
  group('Calculator', () {
    final calculator = Calculator();
```

```
test('add() returns the correct sum', () {  
  // Arrange & Act  
  final result = calculator.add(2, 3);  
  // Assert  
  expect(result, 5);  
});  
  
test('calculateDiscount() applies percentage correctly', () {  
  final result = calculator.calculateDiscount(200, 10);  
  expect(result, 180);  
});  
});  
}
```

This test file can be run from the terminal using:

```
flutter test test/calculator_test.dart
```

REAL-WORLD EXAMPLE: E-Commerce Discount Logic

A retail app such as Daraz or an online grocery app needs to calculate discounts, taxes, and shipping fees correctly every single time, since even a small miscalculation can cause real financial loss or customer complaints. Developers write unit tests for the pricing engine so that every time the code is updated (for example, when a new tax rule is added), the test suite immediately reveals whether the calculation logic still produces correct totals.

3. Widget Testing in Flutter

DEFINITION: Widget Test

A widget test (also called a component test) verifies that a single widget's UI renders correctly and responds correctly to user interaction, such as taps or text entry. It runs in a simulated widget tree without requiring a real device, emulator, or physical screen, making it much faster than a full integration test while still checking actual UI behaviour.

Widget tests use the `WidgetTester` class provided by `flutter_test`. The tester can pump (render) a widget into a virtual environment, simulate user gestures such as `tap()` or `enterText()`, and then verify the resulting UI using finder methods such as `find.text()` or `find.byType()`.

3.1 Key Concepts in Widget Testing

Concept	Purpose
<code>testWidgets()</code>	The entry point for writing a widget test, similar to <code>test()</code> but with a <code>WidgetTester</code> .

Concept	Purpose
<code>pumpWidget()</code>	Builds and renders the given widget into the test environment.
<code>pump()</code> / <code>pumpAndSettle()</code>	Triggers a new frame after a state change or waits for all animations to finish.
<code>find.text()</code> / <code>find.byType()</code>	Locates widgets in the rendered tree so they can be checked or interacted with.
<code>expect()</code>	Asserts that a widget exists, has certain text, or matches an expected condition.

3.2 Code Example: Widget Test

Consider a simple counter widget, similar to the default Flutter starter app:

```
// lib/counter_widget.dart
class CounterWidget extends StatefulWidget {
  @override
  _CounterWidgetState createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int _count = 0;

  void _increment() {
    setState(() => _count++);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(children: [
        Text('Count: $_count'),
        ElevatedButton(
          onPressed: _increment,
          child: Text('Increment'),
        ),
      ]),
    );
  }
}
```

The widget test for this component verifies that tapping the button updates the displayed count:

```
// test/counter_widget_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:my_app/counter_widget.dart';

void main() {
```

```
testWidgets('Counter increments when button is tapped', (WidgetTester tester) async {
  // Arrange: render the widget
  await tester.pumpWidget(MaterialApp(home: CounterWidget()));

  // Assert initial state
  expect(find.text('Count: 0'), findsOneWidget);

  // Act: simulate a tap on the button
  await tester.tap(find.byType(ElevatedButton));
  await tester.pump(); // rebuild the widget after state change

  // Assert updated state
  expect(find.text('Count: 1'), findsOneWidget);
});
}
```

REAL-WORLD EXAMPLE: Login Form Validation

A banking or food-delivery app (for example, a Foodpanda-style app) typically disables its Login button until both the email and password fields contain valid input. A widget test can simulate typing an invalid email, tapping Login, and verifying that an error message such as 'Enter a valid email' appears on screen — confirming the validation logic and the UI work together correctly without needing to install the app on a physical phone.

3.3 Unit Testing vs Widget Testing

Aspect	Unit Testing	Widget Testing
Scope	Single function/class logic	Single widget's UI and interaction
UI Involved?	No	Yes (simulated, not real device)
Speed	Fastest	Fast, but slower than unit tests
Tooling	test package	flutter_test + WidgetTester
Typical Use	Business logic, calculations, models	Buttons, forms, screens, navigation

4. Debugging in Flutter

Debugging is the process of locating, analysing, and fixing defects (bugs) in code. Flutter provides strong debugging support through DevTools, the Dart Debugger, and detailed console logging (`print()` statements, `debugPrint()`, and the Flutter Inspector for visualising the widget tree). Combining good debugging habits with a strong automated test suite is what allows a development team to ship reliable applications confidently and quickly.

- Flutter DevTools — a suite of performance and debugging tools, including a widget inspector, timeline view, and memory profiler.
- Breakpoints — pausing program execution at a specific line to inspect variable values, available in IDEs such as VS Code and Android Studio.
- Hot Reload — allows developers to see the effect of a code change almost instantly without losing the current app state.

Lecture 20: Testing in React Native

Topic: Jest and React Native Testing Library

1. Introduction to Testing in React Native

React Native applications are built using JavaScript and React components, so the React Native ecosystem relies on the wider JavaScript testing ecosystem rather than a bespoke testing toolkit. The two most widely used tools for testing React Native apps are Jest, a JavaScript testing framework, and React Native Testing Library (RNTL), a library built specifically for testing components the way an actual user would interact with them.

2. Jest: The JavaScript Testing Framework

DEFINITION: Jest

Jest is a JavaScript testing framework developed by Meta (the same organisation behind React and React Native). It provides a test runner, an assertion library, and a mocking framework all in one package, and it comes pre-configured in every project created with the React Native CLI or Expo, requiring little to no extra setup.

2.1 Core Features of Jest

Feature	Description
Test Runner	Automatically finds and executes files ending in .test.js or .spec.js.
Assertions (expect)	Provides matchers such as toBe(), toEqual(), toContain() to check expected results.
Mocking	Replaces real modules (e.g., network calls, AsyncStorage) with fake versions for isolated testing.
Snapshot Testing	Captures a rendered component's output and compares it against a saved reference to detect unintended UI changes.
Code Coverage	Reports what percentage of the codebase is exercised by the test suite (jest --coverage).

2.2 Code Example: Jest Unit Test

Consider a simple utility function used to calculate a shopping cart total:

```
// utils/cart.js
export function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}
```

The Jest test file verifies this function behaves correctly:

```
// utils/cart.test.js
import { calculateTotal } from './cart';

describe('calculateTotal', () => {
  it('returns the correct total for multiple items', () => {
    const items = [
      { price: 100, quantity: 2 },
      { price: 50, quantity: 1 },
    ];
    expect(calculateTotal(items)).toBe(250);
  });

  it('returns 0 for an empty cart', () => {
    expect(calculateTotal([])).toBe(0);
  });
});
```

Tests are run from the terminal using the command:

```
npx jest
```

REAL-WORLD EXAMPLE: Ride-Hailing Fare Calculation

A ride-hailing app such as Careem or Uber calculates fares based on distance, time, and surge pricing multipliers. Engineers write Jest tests for the fare calculation function so that any change to the pricing algorithm — for example, adding a new surge rule during Eid traffic — can be verified instantly, without needing to manually book test rides.

3. React Native Testing Library (RNTL)

DEFINITION: React Native Testing Library

React Native Testing Library is a library that provides simple, lightweight utilities for testing React Native components by interacting with them the way a real user would — by finding elements through visible text, labels, or roles, and simulating presses or text input — rather than testing internal implementation details such as state variables.

RNTL is typically used together with Jest: Jest provides the test runner and assertions, while RNTL provides the `render()`, `screen`, and `fireEvent` utilities needed to mount and interact with components. This combination mirrors the relationship between Flutter's test package and its `WidgetTester`.

3.1 Guiding Philosophy

RNTL is built around one well-known testing principle, often summarised as:

“The more your tests resemble the way your software is used, the more confidence they can give you.”

This means tests should query the screen the way a user would see it (for example, by the text on a button) instead of reaching into a component's internal state or props, which makes the tests more robust to internal refactoring.

3.2 Code Example: Component Test with RNTL

Consider a simple login button component:

```
// components/LoginButton.js
import React, { useState } from 'react';
import { View, TextInput, Button, Text } from 'react-native';

export default function LoginButton({ onLogin }) {
  const [email, setEmail] = useState('');

  return (
    <View>
      <TextInput
        placeholder='Email'
        value={email}
        onChangeText={setEmail}
        testID='email-input'
      />
      <Button
        title='Login'
        onPress={() => onLogin(email)}
        disabled={!email.includes('@')}
      />
    </View>
  );
}
```

The RNTL test simulates a real user typing an email and pressing Login:

```
// components/LoginButton.test.js
import React from 'react';
import { render, fireEvent, screen } from '@testing-library/react-native';
import LoginButton from './LoginButton';

test('calls onLogin with the entered email when pressed', () => {
  const mockLogin = jest.fn();
  render(<LoginButton onLogin={mockLogin} />);

  // Simulate user typing an email
  fireEvent.changeText(screen.getByTestId('email-input'), 'student@uni.edu');

  // Simulate user pressing the Login button
  fireEvent.press(screen.getByText('Login'));

  // Verify the callback was called with the typed email
  expect(mockLogin).toHaveBeenCalledWith('student@uni.edu');
});
```

REAL-WORLD EXAMPLE: Social Media Post Button

An app such as Instagram or a university social platform disables its 'Post' button until the caption field has at least one character. Developers use RNTL to simulate a user typing a caption and tapping Post, confirming the button enables correctly and the right function is triggered — exactly as a real user would experience it during manual testing, but automated and repeatable.

3.3 Comparing Flutter Testing and React Native Testing

Aspect	Flutter	React Native
Unit Testing Tool	test (Dart)	Jest
Widget/Component Testing Tool	flutter_test (WidgetTester)	React Native Testing Library
Simulating Taps	tester.tap()	fireEvent.press()
Finding Elements	find.text(), find.byType()	screen.getByText(), getByTestId()
Assertion Style	expect(actual, matcher)	expect(actual).matcher()

Week 11

Lecture 21: App Deployment

Topic: Preparing Apps for Deployment — Building APKs (Android) and IPAs (iOS)

1. What Is App Deployment?

DEFINITION: App Deployment

App deployment is the process of preparing a mobile application for release and distributing it to end users, typically through an official app store. It involves compiling the application's source code into a platform-specific installable package, signing that package with a digital certificate to verify the developer's identity, and submitting it through the appropriate distribution channel.

Deployment is the bridge between development and the real world: it transforms source code that only runs on a developer's machine into a finished product that millions of users can install on their own devices. Because each mobile platform has its own rules, file formats, and submission process, deployment must be handled separately for Android and iOS.

2. Preparing an App for Deployment

Before building a release package, several preparation steps are essential for both platforms:

- Remove debug code — strip out console logs, test data, and developer-only screens.
- Set the correct app name, icon, and splash screen for production branding.
- Update the version number and build number (e.g., version 1.2.0, build 14), since stores require a unique, incrementing build number for every new submission.
- Configure environment variables, such as switching API URLs from a development/staging server to the production server.
- Run the full automated test suite (unit, widget/component, and integration tests) one final time to catch regressions before release.
- Optimise app size and performance, for example by enabling code shrinking (Android's R8/ProGuard) and removing unused assets.

3. Building an APK for Android

DEFINITION: APK (Android Package Kit)

An APK is the standard file format used by the Android operating system to distribute and install mobile applications. It is essentially a compressed archive (similar to a ZIP file) that bundles the compiled application code, resources, assets, and a manifest file describing the app's permissions and components.

3.1 App Bundles (AAB) vs APK

While APK is the traditional installable format, Google Play now requires new apps to be submitted as an Android App Bundle (AAB) — a publishing format that allows Google Play to generate and serve optimised APKs for each user's specific device configuration (screen size, CPU architecture, language), reducing download size. APKs are still used for direct installation, sideloading, and testing outside the Play Store.

3.2 Code Signing on Android

Every Android app must be digitally signed with a cryptographic key before it can be installed on a device or uploaded to the Play Store. This signature proves the app's authenticity and allows the developer to publish future updates that the system can verify came from the same source.

A signing key is generated using the keytool utility:

```
keytool -genkey -v -keystore my-release-key.keystore \  
-alias my-key-alias -keyalg RSA -keysize 2048 -validity 10000
```

3.3 Building the APK

For a Flutter application, the release APK is generated with a single command:

```
flutter build apk --release
```

To produce the Play-Store-preferred App Bundle format instead:

```
flutter build appbundle --release
```

For a React Native application, the release APK is generated through Gradle from the android/ directory:

```
cd android  
./gradlew assembleRelease  
# Output: android/app/build/outputs/apk/release/app-release.apk
```

REAL-WORLD EXAMPLE: Releasing a University Event App

A student-built event management app for a university tech festival, once tested, would be compiled with `flutter build appbundle --release`, signed with a keystore that the development team keeps securely backed up, and only then uploaded to the Play Console — ensuring every future update can be verified as coming from the same trusted developer account.

4. Building an IPA for iOS

DEFINITION: IPA (iOS App Store Package)

An IPA is the file format used to distribute application software on Apple's iOS and iPadOS operating systems. Like an APK, it is a compressed archive containing the compiled app binary, resources, and metadata, but it must be built and signed using Apple's own toolchain, which requires a Mac computer running Xcode.

4.1 The Apple Developer Program and Certificates

Unlike Android, building and distributing an iOS app requires an active Apple Developer Program membership (an annual paid subscription). Through this program, developers obtain signing certificates and provisioning profiles, which together authorise an app to run on specific devices or to be distributed through the App Store.

Requirement	Purpose
Apple Developer Account	Required to access App Store Connect and code-signing tools.
Signing Certificate	Verifies the developer's identity to Apple and iOS devices.
Provisioning Profile	Links the app, the certificate, and approved devices/App Store distribution together.
Xcode	Apple's IDE, required to archive and export the final IPA file.

4.2 Building the IPA

For a Flutter application, the iOS release build is generated with:

```
flutter build ios --release
```

After this command, the project is opened in Xcode, where the developer selects Product > Archive to create an archive, and then uses the Organizer window to export a signed IPA file ready for distribution.

For a React Native application, the equivalent process uses Xcode directly on the .xcworkspace file generated in the ios/ folder, or the command line tool xcodebuild:

```
cd ios
xcodebuild -workspace MyApp.xcworkspace \
  -scheme MyApp -configuration Release \
  -archivePath build/MyApp.xcarchive archive
```

4.3 APK vs IPA: Side-by-Side Comparison

Aspect	APK (Android)	IPA (iOS)
Required OS to Build	Windows, macOS, or Linux	macOS only (via Xcode)
Developer Account Cost	One-time \$25 registration fee	Paid annual membership (~\$99/year)
Signing Tool	keytool / Gradle signing config	Xcode certificates & provisioning profiles
Distribution Formats	APK and AAB (App Bundle)	IPA only
Sideloaded Allowed?	Yes, with user permission	Restricted; limited to development/testing

Lecture 22: Publishing Apps on Google Play and Apple App Store

Topic: Store Submission, Review, and Release Management

1. Introduction to App Store Publishing

Building a signed APK/AAB or IPA is only half the journey — the app must then be submitted to the relevant app store, pass a review process, and be configured with all the metadata (descriptions, screenshots, pricing, privacy policy) that users see before downloading. Each store has its own dashboard, policies, and review timeline.

2. Publishing on Google Play (Google Play Console)

DEFINITION: Google Play Console

Google Play Console is the web-based dashboard Android developers use to publish, manage, and monitor their apps on the Google Play Store, including handling releases, viewing crash reports, managing user reviews, and configuring pricing and distribution.

2.1 Steps to Publish on Google Play

4. Create a Google Play Developer account (one-time registration fee, approximately \$25).
5. Create a new app entry in the Play Console and complete the app's store listing: title, short and full description, screenshots, feature graphic, and category.
6. Complete the Content Rating questionnaire to receive an official age rating.
7. Provide a Privacy Policy URL, mandatory for nearly all apps that collect any user data.
8. Upload the signed AAB (App Bundle) file under the Production release section.
9. Set pricing and country/region availability (free, paid, or in-app purchases).
10. Submit the release for review; Google's automated and manual review process typically takes a few hours to a few days.
11. Once approved, the app becomes available on the Play Store, and updates can be pushed by uploading new, higher-versioned AAB files.

2.2 Release Tracks

Google Play allows apps to be released gradually through different tracks before reaching all users, which reduces the risk of releasing a broken update to everyone at once:

Track	Purpose
Internal Testing	Small group (e.g., the dev team) for quick sanity checks.
Closed Testing	A defined group of testers, such as beta users invited by email.

Track	Purpose
Open Testing	Any user can opt in to test a beta version publicly.
Production	The fully public release available to all Play Store users.

REAL-WORLD EXAMPLE: Staged Rollout for a Banking App

A digital banking app would typically use a staged production rollout — releasing an update to just 10% of users first, monitoring crash reports and reviews in the Play Console for 24–48 hours, and only then expanding to 100% of users, minimising the impact if an unexpected bug appears.

3. Publishing on the Apple App Store (App Store Connect)

DEFINITION: App Store Connect

App Store Connect is Apple's web-based platform for managing apps distributed on the Apple App Store, where developers configure app metadata, upload builds via Xcode or Transporter, manage TestFlight beta testing, and submit apps for Apple's review.

3.1 Steps to Publish on the Apple App Store

12. Enroll in the Apple Developer Program (paid annual membership).
13. Create an App ID and register the app in App Store Connect.
14. Complete the app's store listing: name, subtitle, description, keywords, screenshots for each supported device size, and category.
15. Complete Apple's App Privacy questionnaire, declaring exactly what data the app collects and how it is used.
16. Build and archive the app in Xcode, then upload the signed IPA using Xcode Organizer or the Transporter app.
17. Optionally distribute the build to testers first through TestFlight, Apple's official beta testing platform.
18. Submit the build for App Review; Apple's manual human review typically takes about 24–48 hours, though it can vary.
19. Once approved, release the app immediately, schedule a release date, or release it manually at a chosen time.

3.2 Common Reasons for App Store Rejection

- App crashes or contains obvious bugs during the reviewer's testing.
- Incomplete or misleading metadata, such as screenshots that do not match the actual app.

- Missing or inadequate privacy policy, especially for apps that collect personal data or use location services.
- Use of private or undocumented Apple APIs.
- Broken links, placeholder content, or non-functional features still present in the submitted build.

3.3 Google Play vs Apple App Store: Comparison

Aspect	Google Play Store	Apple App Store
Review Type	Mostly automated, fast	Manual human review, stricter
Typical Review Time	A few hours to 1–2 days	Around 24–48 hours, can be longer
Registration Fee	One-time ~\$25	Annual ~\$99
Beta Testing Tool	Internal/Closed/Open testing tracks	TestFlight
File Format Required	AAB (App Bundle)	IPA
Update Rollout Control	Staged/percentage rollout available	Phased release available (7-day ramp)

4. Post-Publishing Responsibilities

Publishing an app is not the end of the development lifecycle — it is the beginning of an ongoing maintenance cycle. Developers must monitor crash reports, respond to user reviews, track analytics, and release timely updates to fix bugs, support new OS versions, and add features, all while repeating the testing-and-deployment cycle covered in this unit for every new release.

Assignment No. 3

Theoretical Assignment — Based on Lectures 19 to 22

Instructions

- This is a theoretical assignment. No coding submission is required unless explicitly stated in a question.
- Answer all questions in your own words. Direct copying from the internet or classmates will be marked as plagiarism and penalised accordingly.
- Where a code example is requested, you may write pseudocode or simplified Dart/JavaScript syntax — correctness of logic matters more than exact syntax.
- Submit your answer as a single Word or PDF document, with each answer clearly numbered to match the question.
- Total Marks: 50 | Submission Format: Word/PDF document | Weightage: As per course policy

Section A: Short Conceptual Questions (5 marks each — 20 marks)

1. Define unit testing and widget testing in Flutter. Explain, with one example each, how they differ in scope and purpose.
2. Explain the role of Jest and React Native Testing Library in testing a React Native application. Why are these two tools typically used together rather than just one?
3. What is the difference between an APK and an AAB (Android App Bundle)? Why does Google Play now prefer the AAB format for new app submissions?
4. Describe, in your own words, why code signing is a mandatory step before an app can be installed on a real device or submitted to an app store.

Section B: Analytical Questions (10 marks each — 20 marks)

1. Compare the Flutter testing toolkit (unit test + widget test) with the React Native testing toolkit (Jest + RNTL) in a structured table covering: tooling used, how UI elements are located, how user actions are simulated, and overall testing philosophy. Conclude with a short paragraph stating which approach you found more intuitive and why.
2. You are part of a development team preparing a food delivery app for release on both Google Play and the Apple App Store. Describe, step by step, the full deployment pipeline you would follow from the moment development is complete to the moment the app is live on both stores. Your answer should mention preparation steps, building the APK/AAB and IPA, code signing, and the store submission process for each platform.

Section C: Case Study Question (10 marks)

Read the scenario below carefully and answer the questions that follow.

REAL-WORLD EXAMPLE: Case Study: Campus Connect App

A group of final-year students has built 'Campus Connect', a Flutter-based mobile app that lets students view class schedules, register for events, and chat with classmates. The team has finished development and is preparing for their first public release on the Google Play Store. During internal testing, they noticed the chat screen sometimes crashes when a user sends an empty message, and the event registration button does not get disabled after a student has already registered.

1. Identify which type of automated test (unit test or widget test) would have been most useful in catching each of the two bugs described above, and justify your reasoning.
2. List, in the correct order, the deployment and publishing steps the Campus Connect team must complete before their app can go live on the Google Play Store.
3. Suggest which Google Play release track (Internal, Closed, Open, or Production) the team should use first, and explain why a staged approach is safer than releasing directly to Production.

Marking Rubric

Criteria	Marks Allocated
Conceptual accuracy and use of correct terminology	15
Depth of explanation and real-world relevance of examples	15
Logical structuring of deployment/publishing steps	10
Case study reasoning and justification	10

End of Document.